
Lab W02: Galactic Fleet Inventory

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Before You Begin

This assignment must be completed without the use of generative AI tools, including but not limited to ChatGPT, Claude, GitHub Copilot, and similar tools. Write every line of code yourself, using only the lecture material, the textbook, and your own understanding of arrays.

After grading is complete, your code for this lab will be added to a shared class repository so that your classmates can read your solution and you can read theirs. This is standard practice for CIS 3300 and is described in the syllabus. Keep that in mind as you write your code.

2 Overview

You are a supply officer for a small rebel fleet. Fuel, spare parts, and other essentials do not organize themselves, and this lab works through three related problems you will face keeping that fleet running, each one a little more involved than the last.

This lab has three independent parts, each its own Eclipse project:

- Part A works with a single array, summarizing the cargo aboard one ship.
- Part B works with two parallel arrays, searching across a small fleet and flagging ships that need resupply.
- Part C works with a two-dimensional array, treating one ship's cargo bay as a grid and checking it for empty and overloaded slots.

We have not covered classes and objects yet in lecture. You do not need to understand how the provided classes and driver files work internally. For each part, one file is already complete, and it creates an object for you and calls the methods you will write. Your job is to write the body of each method described in that file.

3 Project Setup

Each part of this lab is its own Eclipse project. Repeat the steps below once for each part, using the project name, package name, and files listed in Table 1.

1. Open Eclipse and select File, New, Java Project.
2. Name the project exactly as shown in Table 1 for the part you are starting, then click Finish.
3. In the Package Explorer, expand the new project and right-click the src folder. Select New, Package.
4. Name the package exactly as shown in Table 1, then click Finish.
5. Add the two provided .java files for that part into the new package. There are two ways to do this.
 - **Drag and drop.** Open the folder on your computer that contains the two files, and drag them directly onto the package in the Package Explorer.
 - **Import menu.** Right-click the package, select Import, then General, then File System. Browse to the folder containing the two files, check both files, and click Finish.
6. Confirm both files appear inside the package in the Package Explorer, and that the package line at the top of each file matches the package you just created.
7. Right-click the Driver file for that part, for example CargoHoldDriver.java, and select Run As, Java Application. You should see output using placeholder values, since you have not written any methods yet. This confirms your project is set up correctly before you start coding.

Table 1: Project, package, and file names for each part

Project name	Package name	Files to add
labw02A (Part A)	cargohold	CargoHold.java, CargoHoldDriver.java
labw02B (Part B)	fleettracker	FleetTracker.java, FleetTrackerDriver.java
labw02C (Part C)	cargobay	CargoBay.java, CargoBayDriver.java

4 Comment and TODO Requirements

Two things apply to every file you write in this lab, CargoHold.java, FleetTracker.java, and CargoBay.java. Do not change the three Driver files at all.

4.1 Remove the TODO comments

Every method, and the array declaration comment, currently starts with the word TODO. Once you have written the code for that TODO, delete the TODO line itself. Do not delete the sentence or sentences above it describing what the method or array does, those should remain in your finished file.

4.2 Add a general comment at the top of each class

The comment currently above each `public class` line explains that classes and objects have not been covered yet. That sentence was there to help you while working and should not remain in your finished file. Keep the sentence or sentences describing what the class does, remove that scaffolding sentence, and add the following, per the syllabus, section 4.10.8, General Comments.

- The assignment title
- A version number, starting at 1.0
- Your name as the author

Follow the style shown in Core Java, Volume 1, Listing 3.1, Chapter 3.2. The finished top of each file should look like this.

```
1 package cargohold;
2
3 /**
4  * This class represents the cargo hold of a single ship. It stores
5  * how many units of each supply type are aboard the ship, and
6  * provides methods that summarize that cargo.
7  * Lab W02 Part A: Galactic Fleet Inventory
8  * @version 1.0
9  * @author Jane Student
10 */
11 public class CargoHold {
```

```
1 package fleettracker;
2
3 /**
4  * This class represents a small fleet of ships. Each ship has a
5  * name and a fuel supply, linked by position across two parallel
6  * arrays.
7  * Lab W02 Part B: Galactic Fleet Inventory
8  * @version 1.0
9  * @author Jane Student
10 */
11 public class FleetTracker {
```

```
1 package cargobay;
2
3 /**
4  * This class represents the cargo bay of a single ship, laid out
5  * as a grid of bay levels and storage slots.
6  * Lab W02 Part C: Galactic Fleet Inventory
7  * @version 1.0
8  * @author Jane Student
9  */
10 public class CargoBay {
```

You do not need to worry about any other commenting requirements from the syllabus for this lab, only the general class comment described here.

5 Part A: CargoHold

One ship's cargo hold is represented as an array of supply quantities. You will write methods that summarize that cargo: the total units aboard, the most and least stocked supply type, the average per supply type, and a formatted report that prints all four.

5.1 What is provided

Two files are provided for this part. `CargoHold.java` is where you will write your code, following the TODO comments inside it. `CargoHoldDriver.java` is already complete; it creates a `CargoHold` object and calls each method you write, printing the results.

5.2 What you will write

- `cargoUnits`, an `int` array holding the eight supply quantities listed in the TODO comment in `CargoHold.java`.

Table 2: Methods you will write in `CargoHold.java`

Method	Returns	What it does
<code>getTotalUnits()</code>	<code>int</code>	Sum of every value in <code>cargoUnits</code>
<code>getMostStockedAmount()</code>	<code>int</code>	Largest value in <code>cargoUnits</code>
<code>getLeastStockedAmount()</code>	<code>int</code>	Smallest value in <code>cargoUnits</code>
<code>getAverageUnits()</code>	<code>double</code>	Average value across <code>cargoUnits</code>
<code>printCargoReport()</code>	<code>void</code>	Prints the four values above with clear labels

Once your methods are correct, running `CargoHoldDriver` will print the real values instead of placeholder zeros.

6 Part B: FleetTracker

The fleet now has several ships, each with a name and a fuel level. You will search the fleet for a ship by name, check whether a ship needs resupply, and print a report covering every ship.

6.1 What is provided

Two files are provided for this part. `FleetTracker.java` is where you will write your code, following the TODO comments inside it. `FleetTrackerDriver.java` is already complete; it creates a `FleetTracker` object and calls each method you write, printing the results.

6.2 What you will write

- `shipNames`, a `String` array holding the five ship names listed in the TODO comment in `FleetTracker.java`.
- `fuelSupply`, an `int` array holding the five fuel amounts listed in the TODO comment, kept parallel to `shipNames` by index.

Table 3: Methods you will write in FleetTracker.java

Method	Returns	What it does
findShipIndex(String shipName)	int	Linear search over shipNames; -1 if not found
isBelowThreshold(int shipIndex)	boolean	True if fuelSupply at that index is below resupplyThreshold
printResupplyReport()	void	Loops over the fleet and reports resupply status for every ship

Once your methods are correct, running FleetTrackerDriver will print the real index of Sentinel, the correct threshold result for that ship, and a resupply report covering all five ships.

7 Part C: CargoBay

One ship's cargo bay is now a grid, with bay levels as rows and storage slots as columns. You will count empty slots and check whether any slot is overloaded, then print the grid alongside that summary.

7.1 What is provided

Two files are provided for this part. CargoBay.java is where you will write your code, following the TODO comments inside it. CargoBayDriver.java is already complete; it creates a CargoBay object and calls each method you write, printing the results.

7.2 What you will write

- **bayGrid**, a two-dimensional int array holding the three rows of four values listed in the TODO comment in CargoBay.java.

Table 4: Methods you will write in CargoBay.java

Method	Returns	What it does
countEmptySlots()	int	Counts every zero value in bayGrid using a nested loop
hasOverloadedSlot()	boolean	True if any slot in bayGrid exceeds slotCapacity
printGridReport()	void	Prints the grid row by row, then the results of the two methods above

Once your methods are correct, running CargoBayDriver will print the real empty slot count, the real overload check, and a grid report printed row by row with a correct summary.

8 Grading

This lab is graded using the checklist below. Each item is graded as met or not met, there is no partial credit within a single item, but each item's point value counts toward your total score.

Table 5: Submission and setup, 1 point each

Item	Points
All three parts submitted as a reasonable attempt	1
All three projects compile and run in Eclipse without errors	1
Project and package names match the assignment	1
All three projects committed to GitHub, following the walkthrough video	1

Table 6: Code quality, 1 point each

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
General comment block at the top of each class file, per syllabus 4.10.8	1
TODO comments removed from the finished code	1
No unrequested extra code or output	1

Table 7: Part A, CargoHold, 2 points each

Item	Points
cargoUnits declared correctly with the right values in order	2
getTotalUnits() correct	2
getMostStockedAmount() correct	2
getLeastStockedAmount() correct	2
getAverageUnits() correct	2
printCargoReport() prints all four values with clear labels	2

Table 8: Part B, FleetTracker, 2 points each

Item	Points
shipNames and fuelSupply declared correctly and stay parallel by index	2
findShipIndex() correct, including -1 for not found	2
isBelowThreshold() correct	2
printResupplyReport() correctly reports every ship	2

Table 9: Part C, CargoBay, 2 points each

Item	Points
bayGrid declared correctly with the right dimensions and values	2
countEmptySlots() correct	2
hasOverloadedSlot() correct	2
printGridReport() prints the grid and a correct summary	2

Total: 37 points.

9 Submission

Push all three Eclipse projects to your GitHub repository. A short video walkthrough showing how to commit your projects directly from Eclipse is available in the Course Information module; follow the steps shown there.

Due date: See course LMS for the due date.